MIRA

Entrega 3

Asistente RAG + MCP para el operador de revisión (CU-1)

v3.0	17 pruebas	Recall@4 0,85

Lagos · Moraga · Chaves · Caru

9 de noviembre de 2026

1. Resumen ejecutivo

La tercera entrega extiende MIRA con un asistente para el operador de revisión. Su propósito es explicar por qué un caso llegó a revisión, recuperar reglas verificables y preparar solicitudes de antecedentes. El asistente no aprueba ni rechaza casos: la decisión permanece en una persona.

La solución separa tres responsabilidades. El motor RAG indexa y recupera conocimiento documental desde ChromaDB. El servidor MCP controla el acceso a casos, señales, evidencias, solicitudes y cambios de umbral almacenados en SQLite. La API coordina la conversación y el frontend muestra respuestas, citas, trazas y confirmaciones. Durante la aplicación, la API no abre SQLite; toda lectura o escritura transaccional pasa por MCP.

Se importaron y verificaron 6 documentos, 14 casos, 68 señales, 15 evidencias, 2 solicitudes originales y 2 cambios de umbral. El Golden Set documental contiene 20 preguntas con la distribución exigida: 12 con respuesta, 4 sin respaldo, 2 conflictos vigentes y 2 comparaciones de versiones. A Recall@4, la estrategia por secciones y la de ventanas obtuvieron 0,85 a nivel de evidencia y 0,8125 a nivel de pregunta completa. La suite de integración contiene 17 pruebas automatizadas, todas aprobadas.

Los controles principales bloquean a U-03 por su rol `consulta_indicadores`, impiden solicitudes sobre casos `aprobado_automatico`, reservan la tercera solicitud al supervisor, verifican organización, exigen confirmación humana e impiden duplicados. También se sanitizan RUT, correos, teléfonos, nombres conocidos, credenciales, rutas y claves antes de enviar información a un modelo externo.

Quedan dos actividades externas pendientes: validación del Golden Set por el representante del cliente y evaluación de generación con una clave Gemini habilitada. Estos pendientes no se confunden con fallas de recuperación ni con abstenciones correctas.

2. Especificación del caso de uso

2.1 Objetivo y alcance

CU-1 permite que un operador autenticado consulte un caso de su organización, vea sus señales y evidencias, solicite una explicación basada en reglas y prepare una solicitud de antecedentes. El alcance termina antes de la decisión final. El asistente tampoco modifica umbrales, elimina registros ni interpreta una ausencia de señal como evidencia favorable.

2.2 Actores y flujo principal

1. El operador inicia una sesión con un usuario ficticio del kit.
2. Formula una pregunta en español sobre una regla o un caso.
3. El coordinador solicita al servidor MCP los datos operacionales necesarios.
4. `buscar_reglas` consulta el motor RAG independiente.
5. La respuesta incluye estado, explicación, citas y trazas.
6. Si se prepara una solicitud, la interfaz presenta su contenido exacto.
7. Solo una confirmación explícita ejecuta `solicitar_antecedentes`.
8. Una nueva lectura comprueba el efecto de la acción.

2.3 Principios convertidos en requisitos

Principio	Requisito verificable
PR-1	Ningún mensaje ni herramienta aprueba o rechaza un caso.
PR-2	Toda afirmación normativa incluye fuente, sección, versión y vigencia.
PR-3	Sin respaldo suficiente se declara abstención y la información faltante.
PR-4	Dos fuentes vigentes incompatibles se muestran de forma neutral.
PR-5	Los datos de casos se obtienen por MCP y no se inventan.
PR-6	Toda escritura requiere propuesta y confirmación humana separadas.
PR-7	Organización, rol, estado y límites se validan en el servidor.

2.4 Contradicciones y versiones

Existe un conflicto vigente: POL-DEC v2 §2 fija 85 puntos como umbral de referencia, sujeto a mínimos por señal y configuración organizacional; MAN-REV v1 §1 afirma que 80 puntos o más sin fraude implica aprobación automática. El sistema recupera ambas fuentes, las identifica como vigentes y no elige una por iniciativa propia.

POL-DEC v1 está derogada y se conserva para explicar cambios históricos. En v1 cada señal de fraude descontaba 20 puntos; en v2 el fraude prevalece y deriva el caso. En v1 un diseñador cambiaba umbrales inmediatamente; en v2 se requieren dos supervisores de la organización. En v1 se calculaba con señales disponibles; en v2 una señal faltante obliga a revisión.

3. RAG documental

3.1 Corpus e inventario

El corpus contiene CAT-SEN v3, FAQ-OPE v2, MAN-REV v1, POL-DEC v1 derogada, POL-DEC v2 vigente y PRO-ANT v1. Cada documento se identifica mediante código, versión, estado, fechas de vigencia, archivo fuente y SHA-256. Los archivos permanecen en Markdown para conservar su estructura y facilitar la auditoría.

3.2 Indexación y consulta separadas

`prepare.py` importa el Excel e indexa el corpus. `RagEngine.index` genera colecciones persistentes; `RagEngine.search` las consulta sin acceder a SQLite. El servidor MCP consume ese módulo mediante `buscar_reglas`, pero el motor puede evaluarse directamente sin levantar MCP ni llamar a Gemini.

Se usa `paraphrase-multilingual-MiniLM-L12-v2` en formato ONNX cuantizado y ejecución local. Los vectores y metadatos se almacenan en ChromaDB. Para evitar presión sobre el disco del sistema, modelos, temporales e índices se guardan en B: dentro de la carpeta de Entrega 3.

3.3 Estrategias de fragmentación

Estrategia	Diseño	Fragmentos	Recall@4 pregunta	Recall@4 evidencia
Secciones	Cada encabezado ## conserva la regla completa.	35	0,8125	0,85
Ventanas	450 caracteres, 90 de solapamiento, sin mezclar secciones.	40	0,8125	0,85

Las métricas agregadas empatan, pero los errores no. Secciones no recuperó R05 sobre legibilidad; ventanas sí. Ventanas recuperó una sola fuente en C02, mientras que secciones recuperó ambas. C01 y V02 recuperaron una de dos evidencias esperadas en ambas estrategias. Se mantiene `sections` como predeterminada porque preserva condiciones completas y cubre mejor el conflicto C02 sin aumentar `k`.

3.4 Golden Set y criterio de evidencia

Las 20 preguntas se guardan en `evaluacion/golden_set.json`. Cada registro contiene ID, tipo, texto, evidencia esperada, respuesta esperada y obligación de responder. Las evidencias se definen sobre pasajes del documento original y se mapean a cada estrategia. El commit `4bc95ee` fijó este conjunto antes de la primera evaluación semántica completa.

Las 4 preguntas sin respuesta no participan en Recall porque no tienen fragmentos esperados. Se utilizan para medir abstención durante la evaluación de generación. Los errores del proveedor se clasifican como errores operacionales, nunca como abstenciones correctas.

3.5 Citas, conflictos y abstención

Cada fragmento incluye `chunk_id`, código, título, sección, versión, vigencia, archivo, líneas y texto. La interfaz permite abrir la evidencia citada. Una respuesta sin respaldo debe decir que no existe información suficiente. Para el conflicto 80/85, el modo de demostración ejecutó dos búsquedas controladas y recuperó POL-DEC v2 §2 y MAN-REV v1 §1, ambas vigentes.

4. Servidor MCP y datos operacionales

4.1 Contratos de herramientas

Las herramientas operacionales son `consultar_caso`, `listar_senales`, `consultar_evidencias`, `consultar_solicitudes`, `consultar_cambios_umbral`, `preparar_solicitud_antecedentes` y `solicitar_antecedentes`. `buscar_reglas` delega al RAG. Además existen herramientas técnicas de sesión y trazas que utiliza la API fuera del LLM.

Todas las consultas SQL utilizan parámetros. El servidor resuelve la identidad desde la sesión; ignora cualquier `session_id` o permiso propuesto por el modelo. Una consulta a otra organización devuelve un error genérico sin revelar su organización real.

4.2 Validaciones de escritura

Antes de insertar, `solicitar_antecedentes` revalida dentro de una transacción: sesión, organización, rol, estado, cantidad previa, documentos, motivo, vigencia y pertenencia de la confirmación. U-03 es rechazado porque su rol no es operador ni supervisor. C-010 y C-022 se bloquean por estar aprobados automáticamente. Con dos solicitudes previas, C-016 requiere supervisor.

La confirmación tiene un hash del contenido, usuario, vencimiento y token de un solo uso. Si se repite el mismo token, se devuelve el resultado anterior con `idempotent_replay = true`; no se inserta una segunda solicitud. La escritura agrega una solicitud, un evento de estado y un callback local simulado. La fila original del caso no se modifica.

4.3 Integridad del kit

El importador registra el SHA-256 del Excel: `fd04dc3a40cbbf25e6e9ecf33f2622a86c086f41a71149c49916f3f027e9327f`. También calcula una huella por fila y crea disparadores que bloquean `UPDATE` y `DELETE` sobre registros originales. Las pruebas que escriben regeneran una base aislada desde el kit.

5. Arquitectura, interfaz y privacidad

5.1 Recorrido de una pregunta

La interfaz Next.js llama a FastAPI. La API valida la sesión mediante MCP, sanitiza la pregunta y coordina al proveedor. Gemini —o el proveedor determinista identificado como demo— selecciona herramientas permitidas. El gateway inyecta la sesión verdadera y llama a FastMCP por Streamable HTTP. MCP autoriza el acceso y consulta ChromaDB o SQLite según la capacidad solicitada. La respuesta regresa con citas y trazas visibles.

La API no importa el módulo de base de datos ni abre SQLite. Este ajuste evita que existan dos caminos de acceso operacional y hace comprobable la separación defendida en C5/P3.

5.2 Frontend

La ruta `/asistente` permite elegir usuarios ficticios, iniciar sesión, conversar, identificar estados `evidencia`, `abstencion`, `conflicto` o `error`, abrir citas, inspeccionar trazas y confirmar una acción. El modo offline se rotula explícitamente para no presentarlo como una respuesta de Gemini.

5.3 Datos enviados al modelo

Datos	Tratamiento
Reglas	Solo fragmentos necesarios con código, sección, versión y vigencia.
Casos y señales	IDs ficticios, estado, puntaje, códigos, valores, mínimos y motivos.
Evidencias	Extractos mínimos sanitizados; se consideran datos no confiables.
Nombres, RUT, correos y teléfonos	Enmascarados antes de salir.
Claves, credenciales, rutas y sesión	Excluidos.

La sanitización cubre formatos comunes y nombres conocidos del conjunto. Es una reducción de exposición, no una promesa de anonimización universal. Tampoco se afirma residencia territorial solo por mantener SQLite local.

5.4 Cuotas y fallas

Cada llamada a Gemini pasa por un limitador secuencial configurable. Los errores 429/503 generan reintentos acotados con espera progresiva. El modelo de respaldo se configura por `.env` y sus resultados se identifican por separado. No se usan múltiples claves del mismo proyecto para evadir cuota.

6. Evaluación de sistema

6.1 Resultados automatizados

Se aprobaron 17 pruebas con Pytest. También aprobaron el chequeo TypeScript y la compilación de producción de Next.js. Se verificó un flujo real API → MCP → RAG y un cliente CLI independiente conectado por Streamable HTTP.

Grupo	Evidencias principales
Autorización	U-03 bloqueado; otra organización no revelada; C-010 bloqueado.
Transacción	Tercera solicitud reservada; confirmación idempotente; kit inmutable.
Casos	C-012 umbral 90; C-013 mínimo; C-018 faltante; C-011 aritmética; C-020 idioma.
Seguridad	PII sanitizada; claves excluidas; C-014 tratado como evidencia no confiable.
No efecto	Las lecturas no crean eventos ni solicitudes.

En C-011, la prueba lee E-001 y confirma que 45.000 + 38.000 = 83.000; el total declarado es 93.000 y la diferencia es 10.000. En C-020, DOC_IDIOMA registra portugués y el catálogo indica verificación, no rechazo automático.

6.2 Pruebas manuales y pendientes

La generación con Gemini debe ejecutarse cuando exista una clave disponible y una cuenta sin facturación involuntaria. Esa corrida medirá fidelidad, precisión de citas, abstención correcta e injustificada y manejo de conflictos. El Golden Set debe ser validado por el representante antes de usar sus respuestas como aceptación del cliente.

La caída del proveedor o MCP se presenta como error comprensible y trazable. El cierre manual del servidor puede producir mensajes de cancelación SSE propios de la biblioteca, pero no invalida las llamadas completadas.

6.3 Guion D1–D8 propuesto

D1 abre una cita; D2 demuestra abstención; D3 muestra el conflicto 80/85; D4 exhibe la traza y las versiones; D5 prepara, confirma y relee una solicitud; D6 activa un guardrail; D7 usa el cliente CLI; D8 responde una pregunta imprevista. Este mapeo es una propuesta del equipo basada en las ocho evidencias de la rúbrica, no una nomenclatura desarrollada por el PDF.

7. Riesgos y controles

Riesgo	Control	Evidencia
Inyección en C-014	Evidencia declarada no confiable; instrucciones solo del sistema.	Sin efectos de negocio durante la lectura.
Acceso cruzado	Organización obtenida de sesión y validada en servidor.	U-01 no accede a C-020.
Escritura sin autorización	Rol, estado, límite y confirmación transaccional.	U-03, C-010 y C-016 bloqueados.
Duplicados	Token de un uso e idempotencia.	Repetición devuelve el mismo ID.
Exposición de PII	Sanitización y lista explícita de campos.	Pruebas de RUT, correo, teléfono y nombre.
Cita incorrecta	Metadatos por fragmento y apertura de fuente.	Golden Set y trazas.
Cuota de proveedor	Rate limiting, reintentos, parciales y demo rotulada.	Configuración `.env`.

8. Conclusiones

La entrega demuestra que RAG y MCP resuelven problemas diferentes. RAG aporta evidencia documental trazable y permite comparar estrategias de recuperación. MCP gobierna identidad, permisos, herramientas y efectos sobre datos operacionales. Su integración permite explicar sin transformar al modelo en autoridad de decisión.

El principal aprendizaje técnico fue que no basta con pedir al modelo que sea seguro: las restricciones de organización, rol, estado, cantidad e idempotencia deben existir en el servidor y dentro de la transacción. También fue necesario tratar los conflictos del corpus como información que se muestra, no como un problema que el modelo deba resolver por sí solo.

La versión actual es reproducible y ejecutable en modo offline. Para cerrar la evidencia final se debe: validar el Golden Set con el representante, configurar Gemini, ejecutar la evaluación de generación con control de cuota y reemplazar en el informe esta nota de pendientes por los resultados observados. No se deben inventar métricas ni aprobaciones.

Anexo A. Reproducibilidad

1. Crear el entorno Python e instalar `entrega3/service`.
2. Copiar `.env.example` a `.env`; usar `LLM_PROVIDER=demo` si no hay clave.
3. Ejecutar `python -m mira_cu1.prepare`.
4. Iniciar `python -m mira_cu1.mcp_server` y `python -m mira_cu1.main`.

5. Iniciar el frontend y abrir `/asistente`.
6. Ejecutar `python -m pytest` y `python -m mira_cu1.evaluate --top-k 4`.
7. Probar D7 con `python -m mira_cu1.cli_client consultar_caso --session ... --case C-013`.

Los secretos nunca se incluyen en Git. Los artefactos de ejecución se regeneran en `entrega3/var`.